Лабораторна робота №2

З дисципліни “Системи програмування”

На тему:“Паралелізм данних та паралелізм задач“

Студента 4 курсу: Групи
МІТ-41 Демиденко Андрій

Київ - 2026р.

Завдання 1: Робота з класом Task та синхронізація через WaitAll

Мета завдання: Реалізувати паралельне виконання двох логічних блоків (вивід чисел та літер) за допомогою об'єктів класу Task, використовуючи методи Start() для запуску та WaitAll() для очікування завершення.

Опис роботи програми: Програма базується на класі `MyTask`, який інкапсулює об'єкт `Task` та методи для виводу даних. В основному потоці ініціалізуються дві задачі: перша виконує цикл виводу чисел від 1 до 10, друга — символів від A до J. Використання `Thread.Sleep(200)` дозволяє наочно продемонструвати конкурентне виконання задач у консолі. Метод `Task.WaitAll()` виступає точкою синхронізації, яка блокує основний потік до повного завершення обох асинхронних операцій, що є більш ефективною альтернативою послідовному використанню `Join()`.

Код програми на рисунку №1:

```
namespace Task1
{
    7 references
    class MyTask
    {
        5 references
        public Task Tsk;
        1 reference
        public string TaskName;

        2 references
        public MyTask(string name, Action action)
        {
            TaskName = name;
            Tsk = new Task(action);
        }

        1 reference
        public static void DisplayNumbers()
        {
            for (int i = 1; i <= 10; i++)
            {
                Console.WriteLine($"[Task Numbers] Value: {i}");
                Thread.Sleep(200);
            }
        }

        1 reference
        public static void DisplayLetters()
        {
            for (char c = 'A'; c <= 'J'; c++)
            {
                Console.WriteLine($"[Task Letters] Value: {c}");
                Thread.Sleep(200);
            }
        }
    }

    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            Console.WriteLine("Main thread started.");

            MyTask m1 = new MyTask("NumberTask", MyTask.DisplayNumbers);
            MyTask m2 = new MyTask("LetterTask", MyTask.DisplayLetters);

            m1.Tsk.Start();
            m2.Tsk.Start();

            Task.WaitAll(m1.Tsk, m2.Tsk);

            Console.WriteLine("Main thread completed. All tasks finished.");
        }
    }
}
```

Рисунок №1 — код виконання програми Task1.

Результат виконання на рисунку №2:

```
Main thread started.
[Task Letters] Value: A
[Task Numbers] Value: 1
[Task Numbers] Value: 2
[Task Letters] Value: B
[Task Numbers] Value: 3
[Task Letters] Value: C
[Task Numbers] Value: 4
[Task Letters] Value: D
[Task Numbers] Value: 5
[Task Letters] Value: E
[Task Numbers] Value: 6
[Task Letters] Value: F
[Task Numbers] Value: 7
[Task Letters] Value: G
[Task Letters] Value: H
[Task Numbers] Value: 8
[Task Letters] Value: I
[Task Numbers] Value: 9
[Task Letters] Value: J
[Task Numbers] Value: 10
Main thread completed. All tasks finished.
```

Рисунок №2 — Console output Task1.

Відповідь на питання (Різниця між Thread та Task): Головна відмінність полягає в рівні абстракції та управлінні ресурсами. Клас `Thread` створює реальний потік операційної системи, що вимагає значних витрат пам'яті (~1 МБ на стек) та часу на перемикання контексту. `Task` є високорівневою одиницею роботи, яка використовує системний пул потоків (**ThreadPool**). Це дозволяє виконувати велику кількість задач на обмеженій кількості реальних потоків, що значно підвищує продуктивність системи та масштабованість програмного забезпечення.

Завдання 2: Ідентифікація задач через Id та CurrentId

Мета завдання: Дослідити механізми ідентифікації задач у середовищі виконання .NET, запустивши три паралельні обчислювальні операції.

Опис роботи програми: У даному завданні реалізовано запуск трьох задач, кожна з яких виконує ітераційний підрахунок до 5. Під час кожного кроку циклу в консоль виводяться два типи ідентифікаторів: властивість екземпляра `Task.Id` (унікальний номер об'єкта задачі) та статична властивість `Task.CurrentId` (ID задачі, що виконується в поточному потоці). Програма демонструє, що всередині робочого методу ці значення збігаються, тоді як у головному потоці `Task.CurrentId` повертає порожнє значення (null), оскільки `Main` не є частиною TPL-задачі.

Код програми на рисунку №3:

```
namespace Task2
{
    7 references
    class MyTask
    {
        9 references
        public Task Tsk;
        3 references
        public string Name;

        3 references
        public MyTask(string name)
        {
            Name = name;
            Tsk = new Task(this.Run);
        }

        1 reference
        public void Run()
        {
            for (int i = 1; i <= 5; i++)
            {
                Console.WriteLine($"[{Name}] Object ID: {Tsk.Id} | Current Task ID: {Task.CurrentId} | Step: {i}");
                Thread.Sleep(100);
            }
            Console.WriteLine($"--- {Name} (Id:{Tsk.Id}) finished ---");
        }
    }

    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            Console.WriteLine("Main thread: Starting 3 tasks...");
            Console.WriteLine($"Main thread Task.CurrentId: {Task.CurrentId ?? 0} (null)");

            MyTask mt1 = new MyTask("Alpha");
            MyTask mt2 = new MyTask("Bravo");
            MyTask mt3 = new MyTask("Charlie");

            mt1.Tsk.Start();
            mt2.Tsk.Start();
            mt3.Tsk.Start();

            Task.WaitAll(mt1.Tsk, mt2.Tsk, mt3.Tsk);

            Console.WriteLine("All tasks completed. Main thread finishes.");
        }
    }
}
```

Рисунок №3 — код виконання програми Task2.

Результат виконання на рисунку №4:

```
Main thread: Starting 3 tasks...
Main thread Task.CurrentId: 0 (null)
[Charlie] Object ID: 1 | Current Task ID: 1 | Step: 1
[Bravo] Object ID: 3 | Current Task ID: 3 | Step: 1
[Alpha] Object ID: 2 | Current Task ID: 2 | Step: 1
[Charlie] Object ID: 1 | Current Task ID: 1 | Step: 2
[Bravo] Object ID: 3 | Current Task ID: 3 | Step: 2
[Alpha] Object ID: 2 | Current Task ID: 2 | Step: 2
[Charlie] Object ID: 1 | Current Task ID: 1 | Step: 3
[Alpha] Object ID: 2 | Current Task ID: 2 | Step: 3
[Bravo] Object ID: 3 | Current Task ID: 3 | Step: 3
[Charlie] Object ID: 1 | Current Task ID: 1 | Step: 4
[Alpha] Object ID: 2 | Current Task ID: 2 | Step: 4
[Bravo] Object ID: 3 | Current Task ID: 3 | Step: 4
[Charlie] Object ID: 1 | Current Task ID: 1 | Step: 5
[Alpha] Object ID: 2 | Current Task ID: 2 | Step: 5
[Bravo] Object ID: 3 | Current Task ID: 3 | Step: 5
--- Charlie (Id:1) finished ---
--- Alpha (Id:2) finished ---
--- Bravo (Id:3) finished ---
All tasks completed. Main thread finishes.
```

Рисунок №4 — Console output Task2.

Відповідь на питання (Впорядкованість та відстеження): Ідентифікатори задач у .NET зазвичай призначаються послідовно у порядку створення об'єктів, проте це не є гарантованим правилом, оскільки внутрішні сервіси CLR можуть створювати власні задачі, що перериває нумерацію. Для відстеження виконання ідентифікатори є незамінними: вони використовуються для коректного логування помилок у багатопотоковому середовищі, дозволяють візуалізувати стан задач у вікні "Parallel Tasks" під час дебагу та допомагають розрізняти логічні операції, які можуть виконуватися на одному й тому самому фізичному потоці в різний час.

Завдання 3: Побудова ланцюжків задач через ContinueWith

Мета завдання: Опанувати механізм продовження задач (Continuations) для створення послідовних асинхронних операцій, де результат однієї задачі автоматично передається наступній.

Опис роботи програми: Програма реалізує обчислення суми чисел від 1 до N (де N вводиться користувачем через консоль). Основна логіка побудована на використанні `Task<int>`, яка повертає ціле число після завершення розрахунків. За допомогою методу `ContinueWith` до першої задачі «прив'язується» друга, яка автоматично запускається середовищем виконання одразу після отримання результату. Використання лямбда-виразу дозволяє лаконічно передати об'єкт завершеної задачі (antecedent) у наступну ланку ланцюжка, де через властивість `Result` відбувається вивід фінального значення. Це забезпечує повну асинхронність процесу без потреби в ручному управлінні станом потоків.

Код програми на рисунку №5:

```
namespace Task3
{
    3 references
    class SumCalculator
    {
        3 references
        public int N;

        1 reference
        public SumCalculator(int n)
        {
            N = n;
        }

        1 reference
        public int Calculate()
        {
            Console.WriteLine($"[Task 1] Calculating sum from 1 to {N}...");
            int sum = 0;
            for (int i = 1; i <= N; i++)
            {
                sum += i;
            }
            return sum;
        }
    }

    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            Console.Write("Enter N: ");
            if (!int.TryParse(Console.ReadLine(), out int n)) return;

            SumCalculator calc = new SumCalculator(n);

            Task<int> task1 = new Task<int>(calc.Calculate);
            Task task2 = task1.ContinueWith(antecedent =>
            {
                Console.WriteLine($"[Task 2] Continuation started.");
                Console.WriteLine($"[Result] The total sum is: {antecedent.Result}");
            });

            task1.Start();
            task2.Wait();

            Console.WriteLine("Main thread: All work finished.");
        }
    }
}
```

Рисунок №5 — код виконання програми Task3.

Результат виконання на рисунку №6:

```
Enter N: 6
[Task 1] Calculating sum from 1 to 6...
[Task 2] Continuation started.
[Result] The total sum is: 21
Main thread: All work finished.
marino@marino-82k2:~/My pc/4 курс/System programing$
```

Рисунок №6 — Console output Task3.

Відповідь на питання (Переваги продовжень): Використання продовжень задач має низку суттєвих переваг над класичною синхронізацією. По-перше, це забезпечує неблокуюче виконання: потік, що викликав задачу, не «зависає» в очікуванні, а може виконувати іншу корисну роботу, поки ланцюжок задач обробляється ThreadPool-ом. По-друге, це значно спрощує архітектуру коду,

роблячи його лінійним і читабельним без використання складних примітивів синхронізації (наприклад, `ManualResetEvent`). Крім того, `ContinueWith` дозволяє гнучко налаштовувати умови виконання наступних кроків залежно від успіху, помилки або скасування попередньої задачі, що робить обробку винятків у багатопотокових системах значно надійнішою.

Завдання 4: Паралельне виконання методів через Parallel.Invoke

Мета завдання: Вивчити можливості методу Parallel.Invoke для одночасного запуску декількох різнорідних методів та оцінити ефективність автоматичного планування ресурсів.

Опис роботи програми: У даному завданні реалізовано паралельний запуск трьох функціонально незалежних методів класу `MathWorker`: обчислення факторіалу числа, розрахунок суми ряду від 1 до N та вивід інформаційного повідомлення з примусовою затримкою 300 мс. Використання `Parallel.Invoke` дозволяє запустити всі ці методи одночасно, не створюючи окремі об'єкти Task вручну. Програма демонструє, що методи можуть завершуватися в різному порядку залежно від їхньої складності та затримок, але загальне виконання блоку `Invoke` завершується лише після того, як останній із методів відрапортує про успіх. Це забезпечує зручну «точку збору» для паралельних операцій.

Код програми на рисунку №7:

```
namespace Task4
{
    2 references
    class MathWorker
    {
        1 reference
        public void CalculateFactorial(int n)
        {
            long result = 1;
            for (int i = 1; i <= n; i++) result *= i;

            Console.WriteLine($"[Factorial] {n}! = {result}");
        }

        1 reference
        public void CalculateSum(int n)
        {
            long sum = 0;
            for (int i = 1; i <= n; i++) sum += i;

            Console.WriteLine($"[Sum] Total from 1 to {n} = {sum}");
        }

        1 reference
        public void ShowMessage()
        {
            Console.WriteLine("[Message] Task started...");
            Thread.Sleep(300);
            Console.WriteLine("[Message] 300ms pause finished.");
        }
    }

    0 references
    class Program
    {
        0 references
        static void Main(string[] args)
        {
            MathWorker worker = new MathWorker();

            Console.Write("Enter N for calculations: ");
            if (!int.TryParse(Console.ReadLine(), out int n)) return;

            Console.WriteLine("\n--- Parallel.Invoke starting ---");

            Parallel.Invoke(
                () => worker.CalculateFactorial(n),
                () => worker.CalculateSum(n),
                () => worker.ShowMessage()
            );

            Console.WriteLine("--- Parallel.Invoke finished ---");
            Console.WriteLine("Main thread: All parallel methods completed.");
        }
    }
}
```

Рисунок №7 — код виконання програми Task4.

Результат виконання на рисунку №8:

```
--- Parallel.Invoke starting ---
[Message] Task started...
[Factorial] 6! = 720
[Sum] Total from 1 to 6 = 21
[Message] 300ms pause finished.
--- Parallel.Invoke finished ---
Main thread: All parallel methods completed.
marino@marino-82k2:~/My pc/4 курс/System programing$
```

Рисунок №8 — Console output Task4.

Відповідь на питання (Переваги Parallel.Invoke): Метод `Parallel.Invoke`

є надзвичайно ефективним інструментом у випадках, коли необхідно виконати кілька незалежних операцій паралельно з мінімальними витратами на написання коду. Порівняно зі створенням окремих задач, `Parallel.Invoke` пропонує вбудований механізм очікування завершення всіх операцій (implicit wait), що позбавляє розробника потреби вручну збирати масиви задач та викликати `WaitAll()`. Крім того, цей метод використовує оптимізоване планування, за якого потік, що викликав метод (calling thread), також може брати участь у виконанні однієї з задач, що зменшує накладні витрати на перемикання контексту та створення нових потоків у системному пулі.

Висновок:

У ході виконання лабораторної роботи №2 було досліджено механізми паралельного програмування за допомогою бібліотеки Task Parallel Library (TPL). Використання абстракції Task замість класичних потоків дозволило оптимізувати використання ресурсів через системний пул потоків (ThreadPool), значно знижуючи накладні витрати на керування обчисленнями. Аналіз ідентифікаторів Id та CurrentId підтвердив ефективність моніторингу задач у багатопотоковому середовищі. Реалізація асинхронних ланцюжків через метод ContinueWith та використання об’єкта-антецедента забезпечили неблокуючу передачу даних між етапами обробки, підвищуючи загальну чутливість програми. Застосування Parallel.Invoke продемонструвало переваги автоматичного планування навантаження на багатоядерні архітектури, що дозволяє виконувати різнорідні методи з мінімальними затримками. Отже, перехід до моделі TPL забезпечує створення продуктивного та масштабованого програмного забезпечення завдяки високому рівню абстракції та вбудованим механізмам синхронізації.